

Solicitante: I.T.S - Instituto Tecnológico Superior Arias Balparda.

Nombre Fantasía, de la nueva empresa: Lidenskap

Grupo: 3ºMK

Turno: Matutino

Unidad Curricular: Tecnologías de la información

Nombres de los integrantes del grupo: Sofía Prieto, Federico Rodríguez, Gastón Visos, Joaquín Villena.

Fecha de entrega:28/10/2026

Instituto Tecnológico Superior Arias Balparda.
Blvr. José Batlle y Ordóñez 3570 esq. Gral. Flores – Montevideo.

1. Descripción del sistema..... 4
2. Identificación de amenazas específicas para su sistema..... 5
Amenaza..... 5
Activo afectado:..... 5
Ataque/..... 5
Descripción..... 5
Posible consecuencia..... 5
3. Posibles vulnerabilidades en infraestructura..... 8
4. Controles de acceso y roles de usuarios..... 9
Roles (aplicación)..... 9
Controles a implementar sobre esta capa:..... 9
Usuarios del sistema operativo (servidor):..... 9
5. Definición de políticas de contraseñas..... 10
6. Políticas y medidas de seguridad..... 11
7. Matriz de análisis de seguridad..... 12
1. Introducción..... 13
2. Instalación de Dependencias y Cortafuegos (UFW)..... 13
3. Implementación del Backend (server.js)..... 14
4. Estructura de la Interfaz de Usuario (index.html)..... 15
5. Pruebas de Funcionamiento y Evidencias Prácticas..... 15
6: Diagrama de Red y Flujo de Seguridad..... 17
Código fuente completo..... 18
HTML:..... 18
JS:..... 19
7. Conclusión..... 20
En resumen, la combinación de seguridad a nivel de red, desinfección de entradas, protección contra ataques repetitivos y criptografía unidireccional consolida una arquitectura sólida y preparada para responder de forma segura ante escenarios de uso real..... 20
BLOQUE A Hashing, cifrado y firma digital implementados en el proyecto..... 21
1. Resumen de implementaciones..... 21
2. Hashing..... 22
2.1 Contraseñas de la aplicación (bcrypt)..... 22
2.2 Contraseñas de las cuentas Linux (SHA-512)..... 22
2.3 Contraseñas de los usuarios de MariaDB..... 23
2.4 Integridad de los respaldos (SHA-256)..... 23
3. Cifrado..... 23
3.1 HTTPS (TLS)..... 23
3.2 SSH..... 23
3.3 Respaldos (AES-256)..... 24
3.4 Lo que no se cifra y cómo se protege..... 24

4. Firma digital..... 24
4.1 Certificado del servidor (CA de Lidenskap)..... 24
4.2 Firma de cada respaldo (RSA 4096)..... 25
4.3 Paquetes del sistema..... 25
4.4 Clave de host de SSH..... 25
5. Flujo de un respaldo..... 25
6. Cómo se comprueba..... 26
BLOQUE D Plan de seguridad web basado en OWASP..... 28
1. Arquitectura del sistema..... 28
2. Metodología..... 29
3. Análisis por riesgo OWASP Top 10..... 29
A01: Pérdida de control de acceso..... 29
A02: Configuración de seguridad incorrecta..... 30
A03: Fallas en la cadena de suministro de software..... 31
A04: Fallas criptográficas..... 31
A05: Inyección (SQL, XSS y CSV)..... 32
A06: Diseño inseguro..... 33
A07: Fallas de autenticación..... 34
A08: Fallas de integridad de software y datos..... 34
A09: Fallas de registro y monitoreo..... 35
4. Matriz resumen de riesgos..... 36
5. Pruebas de seguridad y monitoreo continuo..... 40
BLOQUE C Políticas de seguridad de usuario..... 43
2.1 Roles de la aplicación web..... 44
2.2 Cuentas técnicas del servidor..... 44
3. Contraseñas y autenticación..... 46
3.1 Cuentas de la aplicación..... 46
3.2 Cuentas del servidor..... 47
3.3 Obligaciones de todo usuario..... 47
4. Ciclo de vida de las cuentas..... 48
4.1 Alta..... 48
4.2 Roles por torneo..... 48
4.3 Modificación, suspensión y baja..... 49
4.4 Cuentas del servidor..... 49
5. Control de acceso y sesiones..... 49
5.1 Principios..... 49
5.2 Sesiones..... 50
6. Uso aceptable y responsabilidades..... 51
6.1 Responsabilidades por rol..... 51
6.2 Conductas prohibidas..... 51

7. Protección de datos personales..... 52
7.1 Datos que se guardan..... 52
7.2 Menores de edad..... 52
7.3 Derechos del titular..... 53
8. Reporte de incidentes de seguridad..... 53
9. Concientización, cumplimiento y revisión..... 54
9.1 Concientización..... 54
9.2 Sanciones..... 54

Primera Entrega – Ciberseguridad

1. Descripción del sistema

- ¿Cuál es el objetivo del sistema?

SGDM es una aplicación web que tiene como objetivo brindar una plataforma para la organización, administración y seguimiento de torneos deportivos, gestionando inscripciones y la publicación de puntajes en tiempo real durante las competencias, para que tanto como los organizadores y el público puedan seguir su desarrollo

- ¿Quiénes serán sus usuarios?

Los usuarios a nivel de aplicación se dividen en : Administrador general(que gestiona el sistema en su totalidad),Organizador de torneos (que gestiona los torneos que tiene asignados, las inscripciones y la carga de resultados), Participante (incluye a juveniles) y Usuarios publicos (que consultan calendarios, rankings y resultados sin necesidad de registrarse).

- ¿Qué información almacenará?

El sistema almacenará datos personales de los participantes (nombre, edad/categoría, datos de contacto), credenciales de acceso de los usuarios registrados, información de los torneos (fechas, categorías, equipos, asignación de organizadores), resultados y puntajes de las competencias, y un registro de auditoría de las acciones administrativas realizadas dentro del sistema. Ya que que está manejando información de menores de edad y resultados oficiales de competencias, la confidencialidad e integridad de estos datos son especialmente sensibles.

2. Identificación de amenazas específicas para su sistema

SGDM es una aplicación web de gestión deportiva que administra inscripciones de participantes (incluyendo menores de edad en competencias escolares/juveniles) y publicación de puntajes en tiempo real durante los torneos. Estas características exponen superficies de ataque críticas propias del sector deportivo digital.

enumeramos seis amenazas específicas, su método de ataque y su posible efecto en la continuidad del negocio y la reputación de las ligas deportivas que las utilizan:

Amenaza	Activo afectado:	Ataque/ Descripción	Posible consecuencia
Inyección SQL (SQL Injection)	Base de datos del sistema	Manipulación de campos de formularios (inscripción de participantes, login, carga de resultados) para insertar sentencias SQL no autorizadas y leer, modificar o borrar datos de la base	Filtración de datos personales de participantes (incluyendo menores de edad), alteración de resultados oficiales y pérdida total de confianza de las ligas usuarias en la integridad del sistema.
Cross-Site Scripting (XSS)	Interfaz pública y sesiones de los administradores u organizadores.	scripts maliciosos en campos públicos (nombre de equipo, comentarios de perfil, nombre de torneo) que se ejecutan en el navegador de otros usuarios al visualizar calendarios o rankings públicos.	Robo de sesiones de organizadores o administradores, desfiguración de la interfaz pública y daño reputacional frente a las ligas y espectadores.

Amenaza	Activo afectado:	Ataque/ Descripción	Posible consecuencia
Inyección SQL (SQL Injection)	Base de datos del sistema	Manipulación de campos de formularios (inscripción de participantes, login, carga de resultados) para insertar sentencias SQL no autorizadas y leer, modificar o borrar datos de la base	Filtración de datos personales de participantes (incluyendo menores de edad), alteración de resultados oficiales y pérdida total de confianza de las ligas usuarias en la integridad del sistema.
Denegación de Servicio (DoS / DDoS)	Servidor web y disponibilidad de la plataforma.	Saturación del servidor mediante peticiones masivas, especialmente en momentos críticos como la publicación de resultados en tiempo real de un torneo en curso.	Interrupción del servicio durante eventos deportivos en vivo, incumplimiento de la disponibilidad prometida a los organizadores y pérdida de credibilidad comercial.
Manipulación de datos en tránsito	Comunicaciones de red entre los usuarios y el servidor.	interceptación de comunicaciones no cifradas entre el cliente y el servidor para alterar puntajes, resultados o datos de inscripción mientras viajan por la red.	alteración de resultados oficiales de competencias, generando reclamos, descalificaciones injustas y pérdida de la validez deportiva del torneo.
Automatización de registros falsos (Bots / Credential Stuffing)	cuenta de usuario, los sistemas de autenticación y la base de datos de clientes.	Uso de scripts automatizados para crear cuentas falsas, saturar cupos de inscripción o probar combinaciones de credenciales robadas de otros sitios contra el login del sistema.	Cupos de torneos ocupados por participantes ficticios, datos estadísticos corruptos y posible acceso no autorizado a cuentas reales de participantes u organizadores.

Amenaza	Activo afectado:	Ataque/ Descripción	Posible consecuencia
Inyección SQL (SQL Injection)	Base de datos del sistema	Manipulación de campos de formularios (inscripción de participantes, login, carga de resultados) para insertar sentencias SQL no autorizadas y leer, modificar o borrar datos de la base	Filtración de datos personales de participantes (incluyendo menores de edad), alteración de resultados oficiales y pérdida total de confianza de las ligas usuarias en la integridad del sistema.
Broken Access Control	Funciones y permisos del sistema.	fallas en la verificación de permisos (por ejemplo, modificar el ID de un recurso en la URL o en una petición API) para que un Participante acceda a funciones reservadas a un Organizador o Administrador.	Modificación no autorizada de resultados, torneos o configuraciones críticas, comprometiendo la imparcialidad de las competencias gestionadas.

Estas amenazas son priorizadas ya que atacan directamente los activos más sensibles de SGDM: como los datos personales de adultos y menores, la integridad de los resultados deportivos publicados y la disponibilidad del servicio durante eventos en vivo, donde una caída del sistema puede tener un costo en la reputación inmediato para las ligas que confían en la plataforma.

3. Posibles vulnerabilidades en infraestructura

Con respecto al servidor, se utiliza AlmaLinux instalado sobre una máquina virtual en Proxmox. Actualmente el acceso por SSH no cuenta con protección contra ataques de fuerza bruta, como podría ser Fail2Ban o algún sistema de detección de intrusos. Además, todavía no se implementó HTTPS/TLS, por lo que la información que se transmite viaja en texto plano. Al utilizar la versión Minimal de AlmaLinux se reduce la cantidad de servicios instalados y, por lo tanto, la superficie de ataque; sin embargo, cualquier herramienta de monitoreo o diagnóstico debe instalarse manualmente mediante DNF cuando sea necesaria. También falta configurar la rotación de logs, lo que podría provocar la pérdida de registros importantes o incluso ocupar demasiado espacio en el disco. Por otra parte, la máquina virtual dispone únicamente de 1 vCPU y 4 GB de memoria RAM, por lo que tiene un margen reducido para soportar aumentos de carga o ataques que consuman recursos.

En cuanto a la base de datos, MariaDB mantiene una configuración inicial pensada para equipos con mayores recursos, por lo que sería recomendable ajustar parámetros como los buffers y revisar la configuración de los usuarios. Además, los usuarios de la base de datos todavía no están separados según sus funciones, y sigue existiendo el riesgo de sufrir ataques de inyección SQL. Para disminuir este riesgo, es recomendable utilizar prepared statements en todas las consultas realizadas por la aplicación.

Otro aspecto a tener en cuenta es que Apache, PHP y MariaDB están instalados en la misma máquina virtual dentro de la red interna. Esto implica que, si un atacante consigue vulnerar el servidor web, también tendría la posibilidad de acceder de forma directa a la base de datos. Además, el servidor dispone de un solo adaptador de red, por lo que una falla en este componente dejaría al sistema sin conexión. A esto se suma que el servidor se conecta a la red de la UTU, la cual es compartida por estudiantes y funcionarios, sin una segmentación que separe el tráfico correspondiente al proyecto del resto de la red institucional. Al encontrarse todos los equipos dentro del mismo dominio de red, un dispositivo comprometido en otra área de la institución podría intentar acceder al servidor o incluso interceptar el tráfico dirigido hacia él. Por esta razón, sería conveniente verificar si existe una VLAN destinada exclusivamente al proyecto, de manera que el tráfico relacionado con la administración y el desarrollo permanezca aislado del resto de la infraestructura de la institución.

Por último, actualmente no se cuenta con un sistema de copias de seguridad para el servidor ni para la base de datos. En consecuencia, ante un problema de hardware, un ataque de ransomware o la eliminación accidental de información, los datos de participantes, torneos y resultados podrían perderse de forma definitiva, sin posibilidad de recuperarlos.

4. Controles de acceso y roles de usuarios

Es importante distinguir, que existen dos capas de usuarios completamente independientes en este proyecto: los roles funcionales de la aplicación web (definidos en Ingeniería de Software) y las cuentas del sistema operativo del servidor (definidas en Administración de Sistemas Operativos). Ningún usuario del sistema operativo representa a un rol de la aplicación, ni viceversa.

Roles (aplicación)

ROL-01 Administrador general: Nivel 1, control total, sin restricciones operativas sobre todo el sistema.
ROL-02 Organizador de torneo: Nivel 2, acceso restringido únicamente a los torneos que tiene asignados.
ROL-03 Participante: Nivel 3, solo lectura, no modifica resultados ni torneos, solo su información.
ROL-04 Usuario público: Nivel 4, consulta pública sin autenticación, no modifica información.

Controles a implementar sobre esta capa:

Autorización verificada en el backend, nunca únicamente ocultando botones en el frontend. Un Organizador de torneo solo puede operar sobre torneos donde figura asignado.
Renovación del identificador de sesión al iniciar sesión, para evitar que alguien "robe" una sesión ya autenticada.
Registro de auditoría de acciones administrativas, guardar un historial de quién hizo cada cambio importante y cuándo.

Usuarios del sistema operativo (servidor):

sprieto, frodriguez, gvisos, jvillena, cuentas humanas de nuestro equipo, con sudo otorgado vía el grupo sgdm_admins, donde nunca se comparten credenciales entre integrantes.
svc_sgdm, usuario de servicio de la aplicación, sin sudo y sin shell interactiva (/sbin/nologin), dueño de /var/www/sgdm.

5. Definición de políticas de contraseñas

Se definieron políticas diferentes para las contraseñas de la aplicación del SGDM y para las cuentas del sistema operativo del servidor, debido a que ambas cumplen funciones distintas y, por lo tanto, requieren medidas de seguridad específicas.

En la aplicación del SGDM se establecerá que las contraseñas deberán tener un mínimo de 10 caracteres e incluir letras mayúsculas, minúsculas, números y símbolos. Además, estas se almacenarán utilizando algoritmos de hash seguros, como bcrypt o Argon2, evitando el uso de algoritmos inseguros como MD5 o SHA1. También se define que, cuando un administrador asigne una contraseña a un usuario, éste deberá cambiarla obligatoriamente en su primer inicio de sesión. Asimismo, la cuenta se bloqueará temporalmente luego de cinco intentos fallidos y no será posible reutilizar ninguna de las últimas tres contraseñas utilizadas. Para los roles de Administrador General y Organizador de Torneo, además, se establece que la contraseña deberá renovarse cada 90 días.

En cuanto a las cuentas del sistema operativo del servidor al crear un usuario, el sistema genera automáticamente una contraseña temporal de 12 caracteres mediante openssl rand -base64. Además, el propio sistema operativo almacena dicha contraseña en el archivo /etc/shadow, utilizando SHA-512 como algoritmo por defecto en AlmaLinux. Asimismo, mediante el comando chage -d 0 se obliga al usuario a cambiar esa contraseña en su primer inicio de sesión.

6. Políticas y medidas de seguridad

A partir de las amenazas y vulnerabilidades identificadas anteriormente, se definieron las medidas de seguridad que se aplicarán al proyecto. Estas se organizan en tres áreas principales: la aplicación web, el servidor junto con su infraestructura y las prácticas de trabajo del equipo.

En la aplicación web, se establece que todas las consultas a la base de datos se realizarán mediante sentencias preparadas, las cuales ayudan a prevenir ataques de inyección SQL. Asimismo, los datos ingresados por los usuarios serán validados tanto en el frontend como en el backend, ya que la validación del lado del cliente por sí sola no resulta suficiente. Además, toda la información que se muestre en HTML será escapada correctamente para disminuir el riesgo de ataques XSS.

También se prevé que los formularios encargados de modificar información, como la creación de torneos, la carga de resultados o la administración de usuarios, incorporen tokens anti-CSRF. Del mismo modo, cuando ocurra un error, el sistema mostrará mensajes genéricos al usuario, evitando revelar información interna que pueda comprometer la seguridad de la aplicación.

Con respecto al servidor y la infraestructura, se tiene previsto utilizar un firewall (firewalld) configurado con una política restrictiva, la cual permitirá únicamente los puertos que sean necesarios para el funcionamiento del sistema. Asimismo, se realizarán actualizaciones periódicas del sistema operativo, se implementará la rotación de logs mediante logrotate, se incorporará fail2ban para reforzar la protección frente a intentos de acceso no autorizados y, además, se implementará el uso de SSL/TLS con el objetivo de cifrar las comunicaciones entre los usuarios y el servidor.

Con respecto a la continuidad del servicio, se implementará una política de respaldos, backups periódicos y automatizados tanto de la base de datos como de los archivos de la aplicación, almacenando las copias en una ubicación distinta al servidor de producción y verificando periódicamente que dichas copias puedan restaurarse correctamente.

7. Matriz de análisis de seguridad

Identificamos los activos del sistema, las amenazas que los afectan, las vulnerabilidades asociadas y las medidas de seguridad propuestas.

Activo	Amenaza	Vulnerabilidad	Medida de seguridad
Base de datos (datos personales e información de torneos)	Inyección SQL (SQLi) que borre o altere los resultados	Usuarios sin roles separados en la base de datos	Uso de sentencias preparadas (prepared statements) en todas las consultas a la base de datos
Servidor de la aplicación web e infraestructura	Caída del sistema por Denegación de Servicio (DoS / DDoS)	Recursos de hardware reducidos (1 vCPU, 4GB RAM) y accesos sin protección	Instalación y configuración de Firewalld restrictivo y Fail2Ban
Tráfico de red y comunicaciones de usuarios	Manipulación e interceptación de los datos en tránsito	Transmisión de credenciales y puntajes viajando en texto plano	Despliegue de certificados criptográficos bajo el protocolo SSL/TLS
Interfaz del sistema (Navegador del cliente)	Ataques de Cross-Site Scripting (XSS) y robo masivo de sesiones	Carencia de un adecuado escapado de caracteres en las vistas HTML	Implementación de tokens anti-CSRF y escapado estricto de todas las variables
Accesos y perfiles de la plataforma	Uso de credenciales robadas y automatización de bots (Stuffing)	Falta de un límite de intentos permitidos en la ventana de login	Bloqueo temporal del usuario tras alcanzar cinco intentos fallidos en la autenticación

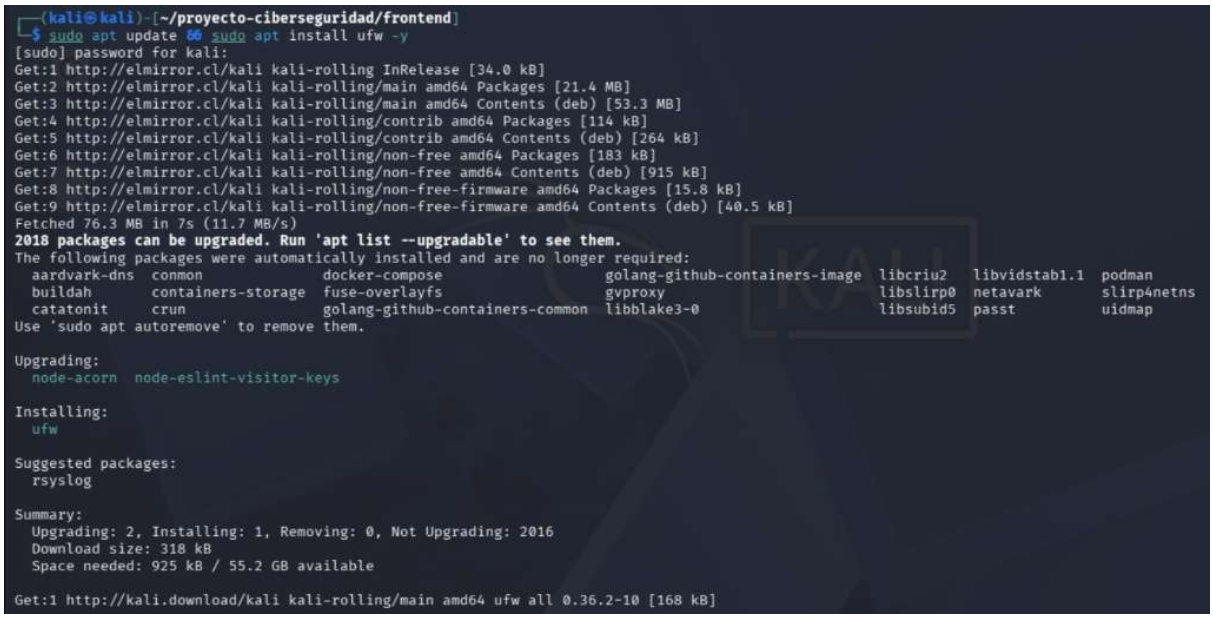
Segunda Entrega Ciberseguridad

1. Introducción

El presente informe documenta la implementación de medidas de seguridad en el desarrollo de un servicio web backend sobre Node.js y Express. Se abordan mecanismos de hardening a nivel de servidor (firewall), control de acceso (limitación de tasa de peticiones), saneamiento e higiene de entradas de datos, y almacenamiento seguro de credenciales mediante hashing unidireccional.

2. Instalación de Dependencias y Cortafuegos (UFW)

Para proteger el sistema a nivel de red y gestionar el tráfico entrante/saliente, se instaló la herramienta ufw (Uncomplicated Firewall). Adicionalmente, se integró el paquete express-validator para validar las entradas recibidas por la API REST.

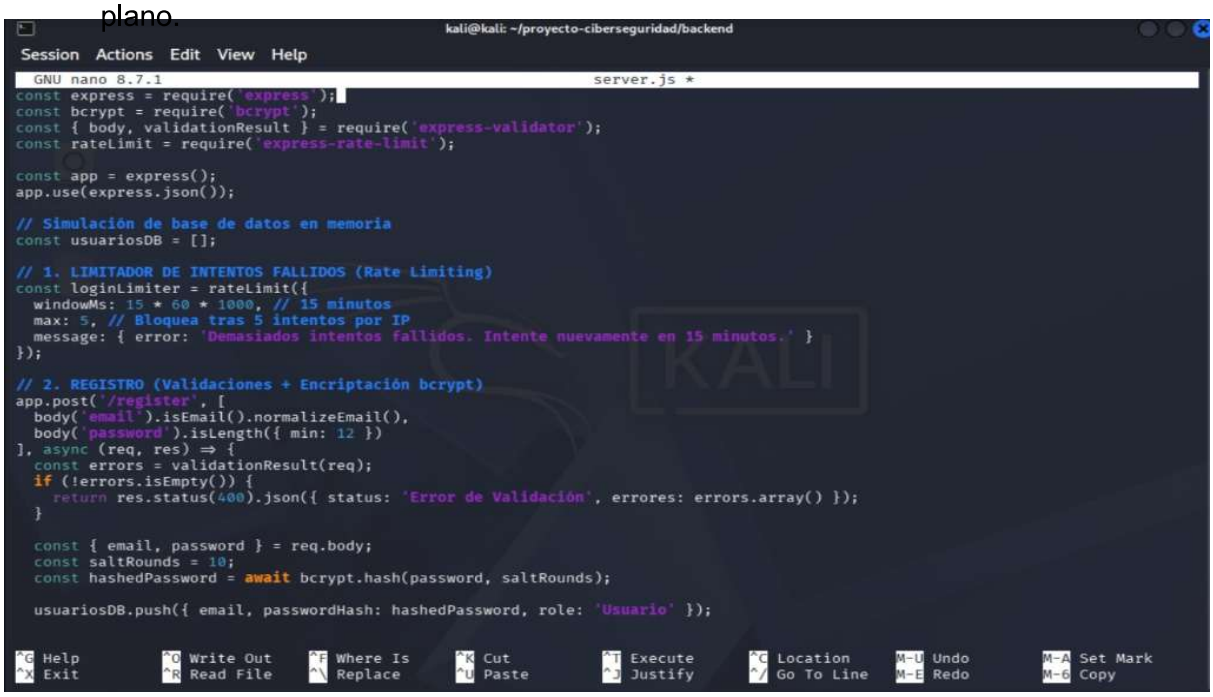


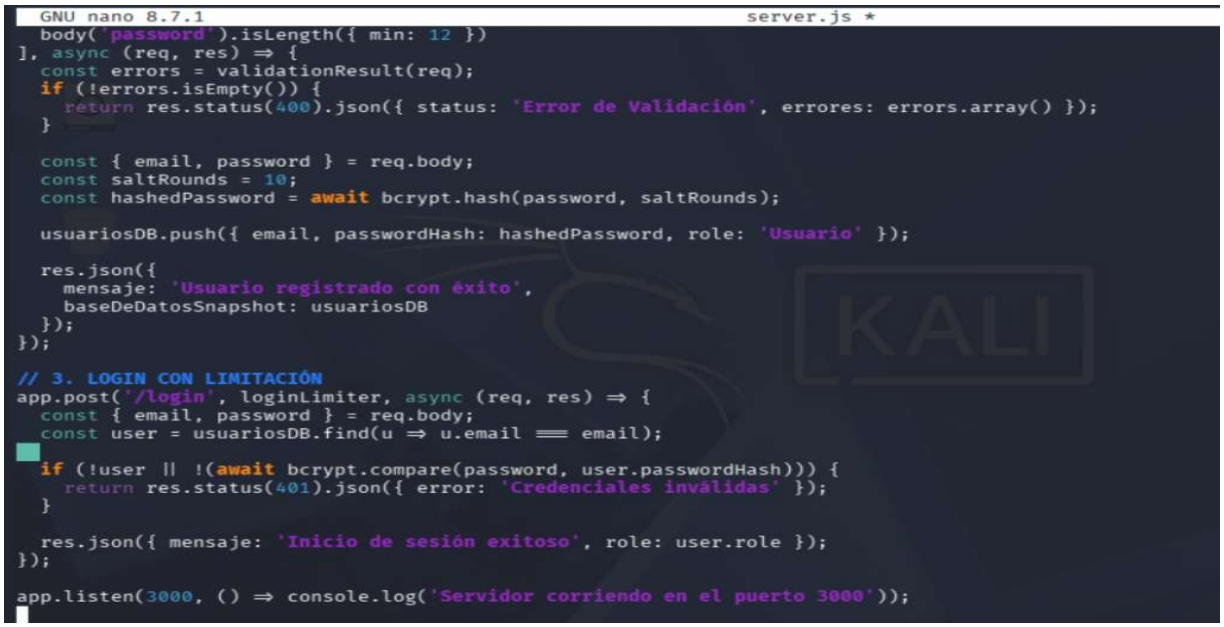


3. Implementación del Backend (server.js)

El servidor principal se desarrolló en la ruta ~/proyecto-ciberseguridad/backend/server.js. Incluye las siguientes características de seguridad:

- Rate Limiting (express-rate-limit): Limita a un máximo de 5 intentos de inicio de sesión fallidos por dirección IP en una ventana de 15 minutos.
- Validación de entradas (express-validator): Normaliza correos electrónicos y exige contraseñas con un mínimo de 12 caracteres.
- Hashing de contraseñas (bcrypt): Aplica un algoritmo de cifrado fuerte con un factor de costo (saltRounds) de 10 para evitar almacenar credenciales en texto plano.





4. Estructura de la Interfaz de Usuario (index.html)

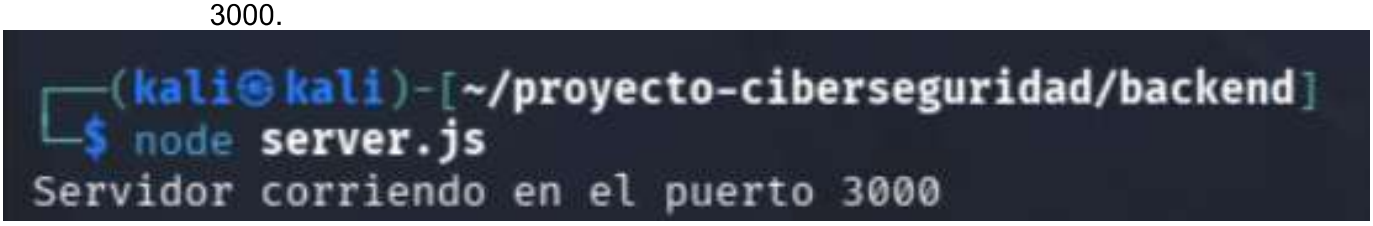
Se implementó un formulario de inicio de sesión básico en el cliente con reglas de validación nativas (type="email", minlength="12", required).

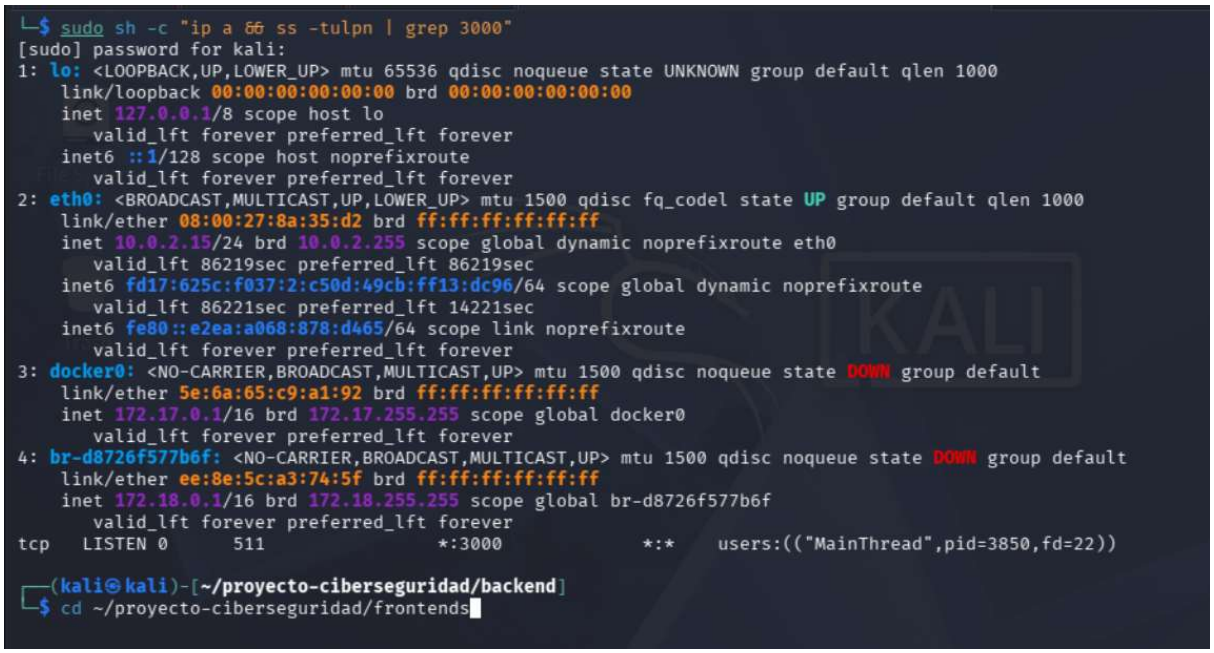


5. Pruebas de Funcionamiento y Evidencias Prácticas

Para comprobar el funcionamiento del backend, se inició el servidor en Node.js y se realizaron las pruebas de autenticación y seguridad mediante la herramienta curl.

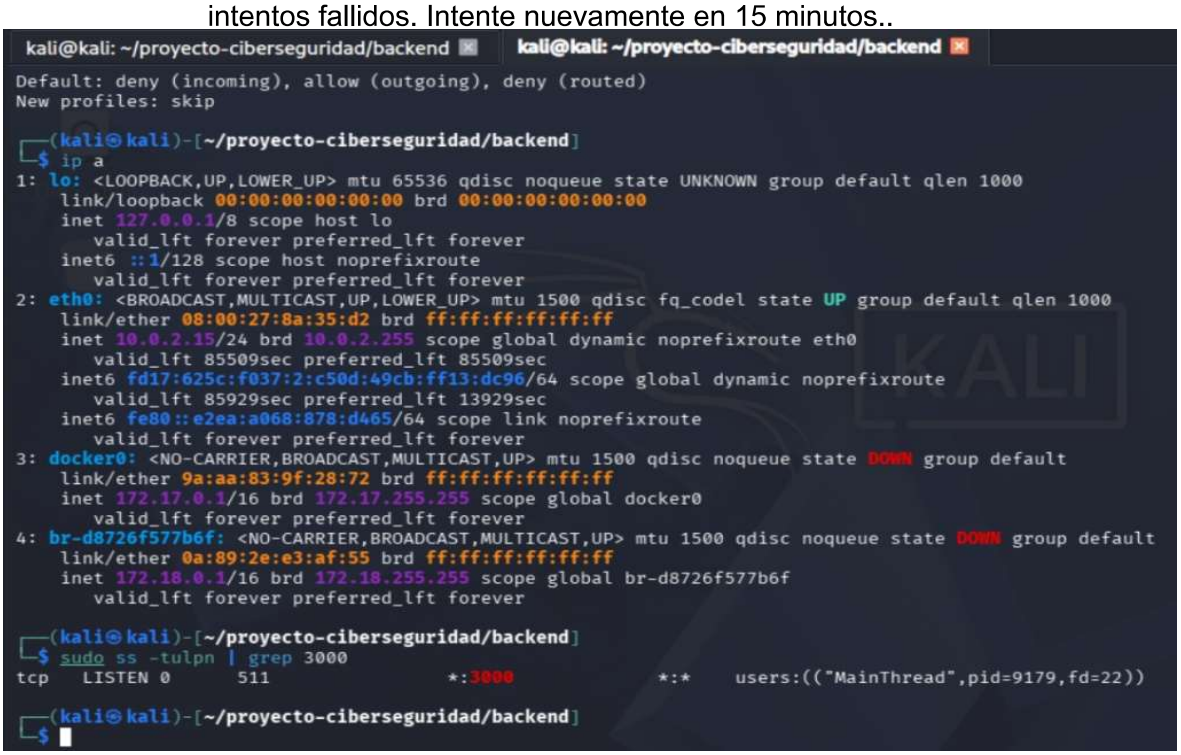
- Captura: Ejecución del servidor Node.js (node server.js) escuchando en el puerto 3000.



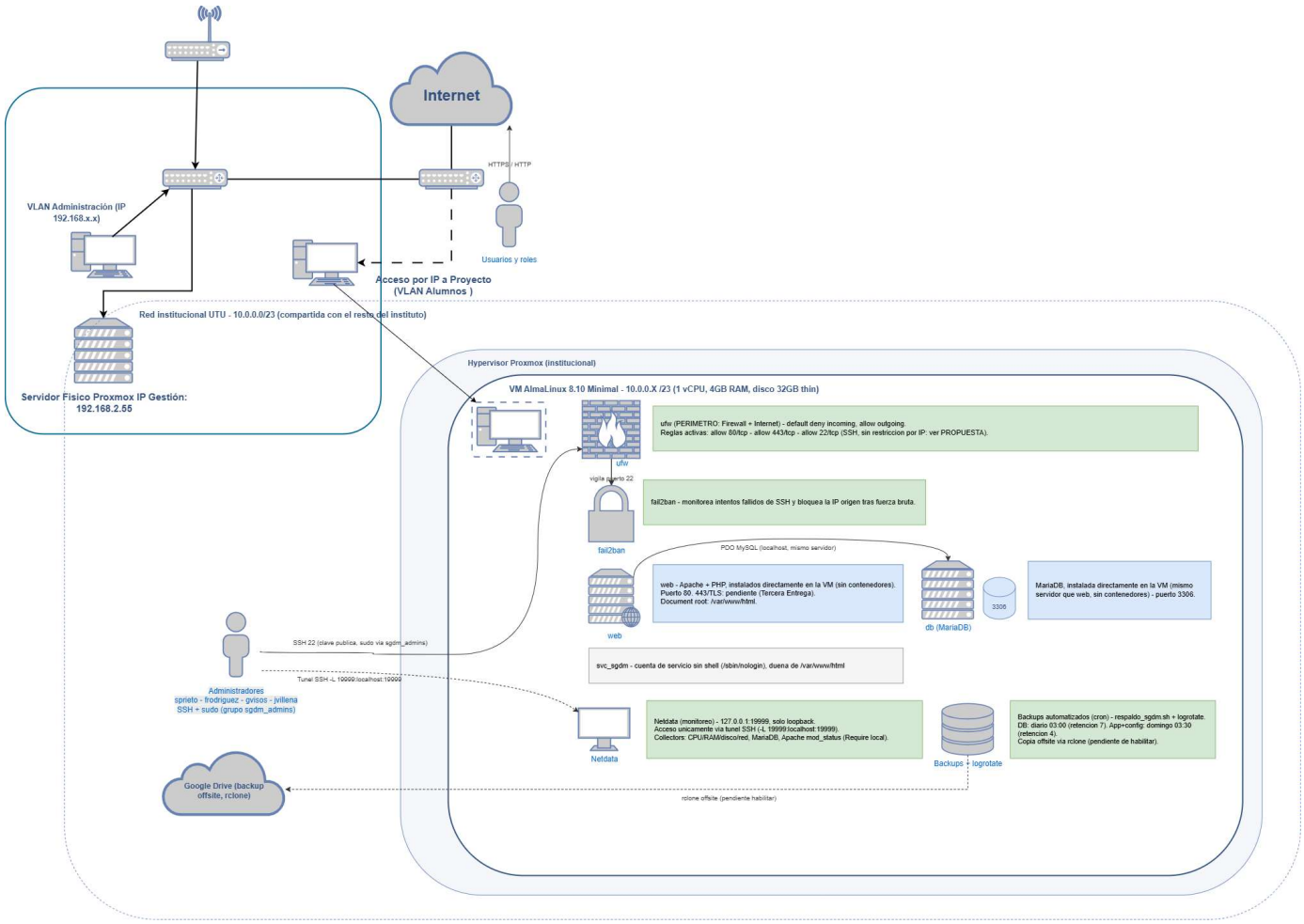


Captura: Demostración de las pruebas de seguridad en terminal:

1. Estado del Firewall: Activación y confirmación de regla predeterminada (Status: active).
2. Registro y Cifrado: Envío exitoso de credenciales a /register, devolviendo el hash seguro generado por bcrypt ($2b$10$...).
3. Bloqueo por Fuerza Bruta (Rate Limit): Ejecución iterativa de 6 peticiones de acceso fallidas. Los primeros 5 intentos devolvieron Credenciales inválidas y el 6.º intento fue denegado con el mensaje Demasiados intentos fallidos. Intente nuevamente en 15 minutos..



6: Diagrama de Red y Flujo de Seguridad



Código fuente completo

HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ciberseguridad - Segunda Entrega</title>
    <style>
        body { font-family: Arial, sans-serif; background: #1a1a1a; color: #fff; display: flex;
justify-content: center; align-items: center; height: 100vh; margin: 0; }
        .card { background: #2a2a2a; padding: 20px; border-radius: 8px; width: 300px; }
        input, button { width: 100%; padding: 10px; margin: 8px 0; box-sizing: border-box; }
        button { background: #007bff; color: white; border: none; cursor: pointer; }
    </style>
</head>
<body>
    <div class="card">
        <h2>Inicio de Sesión</h2>
        <form id="loginForm">
            <input type="email" id="email" placeholder="Correo electrónico" required>
            <input type="password" id="password" placeholder="Contraseña (mín 12
caracteres)" minlength="12" required>
            <button type="submit">Ingresar</button>
        </form>
        <p id="msg"></p>
    </div>
</body>
</html>
```

JS:

```
const express = require('express');
const bcrypt = require('bcrypt');
const { body, validationResult } = require('express-validator');
const rateLimit = require('express-rate-limit');

const app = express();
app.use(express.json());

const usuariosDB = [];

// Limite de peticiones para prevenir fuerza bruta
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Demasiados intentos fallidos. Intente nuevamente en 15 minutos.' }
});

// Ruta de registro con validaciones de entrada y hashing
app.post('/register', [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 12 })
], async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ status: 'Error de Validación', errores: errors.array() });
  }

  const { email, password } = req.body;
  const saltRounds = 10;
  const hashedPassword = await bcrypt.hash(password, saltRounds);

  usuariosDB.push({ email, passwordHash: hashedPassword, role: 'Usuario' });

  res.json({
    mensaje: 'Usuario registrado con éxito',
    baseDeDatosSnapshot: usuariosDB
  });
});

// Ruta de autenticación con limitador de tasa
app.post('/login', loginLimiter, async (req, res) => {
```

```
  const { email, password } = req.body;
  const user = usuariosDB.find(u => u.email === email);

  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: 'Credenciales inválidas' });
  }

  res.json({ mensaje: 'Inicio de sesión exitoso', role: user.role });
});

app.listen(3000, () => console.log('Servidor corriendo en el puerto 3000'));
```

7. Conclusión

Las pruebas prácticas y los resultados obtenidos confirman que la aplicación web implementada cumple de manera efectiva con los estándares esenciales de ciberseguridad y las buenas prácticas en el desarrollo backend.

A través de las distintas etapas de configuración, se lograron mitigar riesgos críticos comunes en aplicaciones expuestas a red:

- Protección del perímetro y servidor: La activación y estructuración del cortafuegos (UFW) asegura un control estricto sobre el tráfico de red, restringiendo accesos no autorizados y sirviendo como una primera línea de defensa a nivel de infraestructura.
- Mitigación de ataques de fuerza bruta: La implementación del control de tasa (*Rate Limiting*) bloquea eficazmente los intentos automatizados de adivinación de credenciales. Al limitar las peticiones fallidas consecutivas por dirección IP, se previene el colapso del servicio por denegación de servicio (DoS) y el filtrado no deseado de cuentas.
- Integridad y validación de datos: El uso de middlewares de validación (express-validator) garantiza que únicamente la información que cumpla con los requisitos de formato (correos válidos y contraseñas con una longitud mínima de 12 caracteres) sea procesada por el servidor, previniendo inyecciones de código y datos maliciosos.
- Almacenamiento seguro de credenciales: Mediante el uso del algoritmo de hashing bcrypt con un factor de costo (*salt*) adecuado, se elimina el almacenamiento de contraseñas en texto plano. Esto garantiza la confidencialidad de las credenciales de los usuarios incluso ante un eventual compromiso de la base de datos.

En resumen, la combinación de seguridad a nivel de red, desinfección de entradas, protección contra ataques repetitivos y criptografía unidireccional consolida una arquitectura sólida y preparada para responder de forma segura ante escenarios de uso real.

Tercera Entrega Ciberseguridad

BLOQUE A Hashing, cifrado y firma digital implementados en el proyecto

Este documento especifica las implementaciones criptográficas del SGDM: qué se protege, con qué algoritmo, en qué parte del proyecto está hecho y cómo se comprueba. Las tres técnicas resuelven problemas distintos:

Técnica	Qué garantiza	¿Se puede revertir?
Hash	Integridad: si el dato cambia un solo bit, el hash cambia. Permite guardar contraseñas sin poder leerlas	No, es de una sola vía
Cifrado	Confidencialidad: solo lee el dato quien tiene la clave	Sí, con la clave
Firma digital	Autenticidad e integridad: el dato lo generó quien tiene la clave privada y nadie lo modificó	No se revierte: se verifica con la clave pública

1. Resumen de implementaciones

Qué se protege	Técnica	Algoritmo	Dónde está implementado
Contraseñas de la aplicación	Hash	bcrypt (costo 10, sal aleatoria)	app/api/register.php, usuarios.php, login.php
Contraseñas de las cuentas Linux	Hash	SHA-512 crypt (6)	/etc/shadow, gestion_usuarios.sh
Contraseñas de los usuarios de MariaDB	Hash	mysql_native_password	02-users.sh, instalar_sgdm.sh
Navegador ↔ servidor	Cifrado	TLS 1.2 / 1.3 (HTTPS)	Apache mod_ssl, ssl_sgdm.sh (Bloque B)
Administración remota	Cifrado	SSH (AES-GCM / ChaCha20)	OpenSSH, seguridad_sgdm.sh
Respaldos (base de datos y aplicación)	Cifrado	AES-256-CBC, clave derivada con PBKDF2	respaldo_sgdm.sh
Respaldos (base de datos y aplicación)	Hash	SHA-256	respaldo_sgdm.sh

Respaldos (base de datos y aplicación)	Firma digital	RSA 4096 + SHA-256	respaldo_sgdm.sh
Certificado HTTPS	Firma digital	RSA 4096 + SHA-256 (CA "Lidenskap CA SGDM")	ssl (Bloque B)
Paquetes del sistema	Firma digital	GPG (RSA) de AlmaLinux	dnf (gpgcheck=1), dnf-automatic

2. Hashing

2.1 Contraseñas de la aplicación (bcrypt)

La contraseña de cada usuario del SGDM nunca se guarda: se guarda su hash bcrypt, en la columna usuario.password.

- **Alta de la cuenta.** Al registrarse, o cuando el Administrador crea una cuenta, se calcula con password_hash($password, PASSWORD_DEFAULT) (api/register.php y api/usuarios.php). En PHP 8.3, PASSWORD_DEFAULT es bcrypt con costo 10 (2^{10} = 1024 rondas).
- **Inicio de sesión.** password_verify($password, $usuario['password']) (api/login.php) recalcula el hash con la sal guardada y compara en tiempo constante.

El resultado tiene la forma $2y$10$ seguido de 22 caracteres de sal y 31 de hash. La sal es aleatoria para cada usuario: dos personas con la misma contraseña tienen hashes distintos, y una tabla de hashes precalculados (rainbow table) no sirve. bcrypt es lento a propósito, por lo que probar contraseñas por fuerza bruta es muy costoso. Si se robara la tabla usuario, las contraseñas no quedarían expuestas.

La columna es VARCHAR(255), y no 60, para que PHP pueda pasar a un algoritmo más nuevo (por ejemplo Argon2) sin cambiar el esquema.

2.2 Contraseñas de las cuentas Linux (SHA-512)

Las contraseñas de los administradores del servidor (grupo sgdm_admins) se guardan en /etc/shadow como hash SHA-512 crypt (6sal$hash, 5000 rondas), el método que define ENCRYPT_METHOD SHA512 en /etc/login.defs de AlmaLinux 8. Solo root puede leer /etc/shadow.

gestion_usuarios.sh crear-admin genera la contraseña temporal con openssl rand -base64 12 (generador aleatorio criptográfico), la carga con chpasswd, que aplica el hash SHA-512, y obliga a cambiarla en el primer ingreso (chage -d 0).

2.3 Contraseñas de los usuarios de MariaDB

sgdm_admin, sgdm_app y sgdm_consulta se crean con IDENTIFIED BY, y MariaDB guarda solo el hash (mysql_native_password). Es un hash viejo (SHA-1 doble, sin sal); por eso el instalador genera contraseñas largas y aleatorias (openssl rand -hex 16, 128 bits) y el puerto 3306 no queda abierto hacia afuera. Las contraseñas en texto plano solo están en .env y en /etc/mysql/backup.cnf, con permisos 600 (solo root).

2.4 Integridad de los respaldos (SHA-256)

Cada respaldo tiene al lado un archivo .sha256 con su hash SHA-256, en el formato estándar de sha256sum. Sirve para detectar un archivo dañado (falla de disco, copia incompleta a Google Drive) y se puede comprobar en cualquier PC sin ninguna clave con sha256sum -c archivo.enc.sha256. El flujo completo está en el punto 5.

3. Cifrado

3.1 HTTPS (TLS)

Todo el tráfico entre el navegador y el servidor viaja cifrado con TLS (Apache con mod_ssl): las contraseñas del login, la cookie de sesión y los datos de los torneos. Las peticiones a http:// se redirigen a https://. Las políticas criptográficas de AlmaLinux 8 (update-crypto-policies --show devuelve DEFAULT) solo permiten TLS 1.2 y 1.3, con cifrados como AES-GCM y claves RSA de 2048 bits o más. La instalación del certificado se detalla en el Bloque B.

En TLS se combinan las tres técnicas. Con el certificado (firma) el navegador comprueba que habla con el servidor correcto; después se acuerda una clave de sesión y con ella se cifra todo con un algoritmo simétrico (AES). Cada mensaje lleva un código de autenticación que detecta cualquier modificación, y las claves de sesión se derivan con funciones de hash (SHA-256 o SHA-384).

La cookie de sesión lleva Secure (solo viaja por HTTPS), HttpOnly (el JavaScript no la puede leer) y SameSite=Lax (app/src/Auth.php).

3.2 SSH

La administración remota del servidor se hace solo por SSH, que cifra toda la sesión, contraseña incluida, con AES-GCM o ChaCha20-Poly1305 según las mismas políticas criptográficas del sistema. Además, seguridad_sgdm.sh ssh impide entrar como root y solo deja entrar al grupo sgdm_admins. Los archivos se copian con scp, que usa el mismo canal cifrado.

3.3 Respaldos (AES-256)

respaldo_sgdm.sh cifra cada respaldo antes de guardarlo y antes de enviarlo a Google Drive. Así, una copia robada o una cuenta de Drive comprometida no exponen la base de datos ni la configuración del servidor.

openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -pass file:/etc/sgdm/respaldos/cifrado.clave

- **AES-256:** cifrado simétrico estándar, con clave de 256 bits.
- **Contraseña de cifrado (cifrado.clave):** 48 bytes aleatorios (openssl rand -base64 48), creados la primera vez que se respalda.
- **PBKDF2:** la clave AES se deriva de la contraseña con 100.000 iteraciones y una sal aleatoria distinta en cada respaldo, por lo que dos respaldos iguales dan archivos cifrados distintos.

Se usa cifrado simétrico porque el mismo servidor cifra y descifra (al restaurar). La clave también tiene que guardarse fuera del servidor, en el gestor de contraseñas del equipo: si se pierde el servidor, sin ella los respaldos de Google Drive no se pueden abrir.

3.4 Lo que no se cifra y cómo se protege

Qué	Por qué no se cifra	Cómo se protege
Base de datos en el disco del servidor	MariaDB la tiene que leer todo el tiempo	Permisos de /var/lib/mysql, firewall y usuarios de base de datos con mínimo privilegio
.env, /etc/mysql/backup.cnf y cifrado.clave	Los programas necesitan leer la contraseña real	Permisos 600, dueño root

El paso siguiente, como en una empresa, sería cifrar el disco completo con LUKS.

4. Firma digital

Una firma digital se hace calculando el hash del dato y cifrándolo con la clave privada. Cualquiera que tenga la clave pública puede verificarla: si el dato cambió o lo firmó otra clave, la verificación falla.

4.1 Certificado del servidor (CA de Lidenskap)

ssl_sgdm.sh crea una autoridad certificadora propia, "Lidenskap CA SGDM" (RSA 4096), que firma el certificado del servidor (sgdm.crt, firma SHA-256 con RSA). Cada PC importa una vez el certificado público de la CA; desde ese momento el navegador verifica la firma y muestra el candado. Además, en cada conexión el servidor firma el intercambio de

claves con su clave privada (sgdm.key), con lo que prueba que es el dueño del certificado. El procedimiento completo está en el Bloque B.

4.2 Firma de cada respaldo (RSA 4096)

respaldo_sgdm.sh firma cada respaldo cifrado con un par de claves propio, separado del de la CA:

```
openssl dgst -sha256 -sign   /etc/sgdm/respaldos/firma.key -out archivo.enc.sig
archivo.enc
openssl dgst -sha256 -verify /etc/sgdm/respaldos/firma.pub -signature archivo.enc.sig
archivo.enc
```

- **firma.key:** clave privada RSA 4096, con permisos 600; no sale del servidor.
- **firma.pub:** clave pública; se puede copiar a cualquier PC para verificar.

El hash SHA-256 detecta daños accidentales, pero quien modifica un respaldo a propósito también puede recalcular el .sha256. La firma no se puede rehacer sin firma.key, así que un respaldo con firma válida lo generó el servidor del SGDM y no fue modificado. Antes de restaurar se verifica la firma, para no cargar en la base un archivo adulterado.

Se firma el archivo ya cifrado (primero cifrar, después firmar): así la copia de Google Drive se puede verificar sin descifrarla.

4.3 Paquetes del sistema

dnf solo instala paquetes con la firma GPG de AlmaLinux (gpgcheck=1 en /etc/yum.repos.d/). dnf-automatic, que instala cada día las actualizaciones de seguridad (seguridad_sgdm.sh), también verifica esa firma, por lo que no se instala software modificado por un tercero.

4.4 Clave de host de SSH

El servidor tiene claves de host (/etc/ssh/ssh_host_*_key). En cada conexión firma el intercambio con ellas, y el cliente compara la clave con la que guardó la primera vez (known_hosts). Si otro equipo se hace pasar por el servidor, SSH muestra "REMOTE HOST IDENTIFICATION HAS CHANGED" y no se conecta.

5. Flujo de un respaldo

Cada respaldo pasa por las tres técnicas y queda formado por tres archivos:

```
mariadb-dump | gzip | AES-256 -> bd_sgdm_FECHA.sql.gz.enc        (cifrado)
sha256sum                  -> bd_sgdm_FECHA.sql.gz.enc.sha256  (hash)
openssl dgst -sign (RSA 4096) -> bd_sgdm_FECHA.sql.gz.enc.sig     (firma)
```

Lo mismo ocurre con el código y la configuración (app_sgdm_…tar.gz.enc). Los tres archivos se guardan en /var/backups/sgdm/bd y /var/backups/sgdm/app (permisos 700) y se envían juntos a Google Drive. La rotación borra los tres a la vez.

Archivo	Qué es	Permisos
/etc/sgdm/respaldos/cifrado.clave	Contraseña de cifrado AES-256 (se guarda una copia fuera del servidor)	600, root
/etc/sgdm/respaldos/firma.key	Clave privada de firma, RSA 4096	600, root
/etc/sgdm/respaldos/firma.pub	Clave pública para verificar la firma	644

Las claves se crean solas en el primer respaldo y no se deben volver a crear: los respaldos anteriores no se podrían descifrar ni verificar. El script tiene estas acciones, también disponibles en el gestor del servidor (menú Respaldos, opciones 5, 6 y 7):

Comando	Qué hace
sudo respaldo_sgdm.sh bd	Respaldo de la base de datos cifrado, con hash y firma
sudo respaldo_sgdm.sh verificar ARCHIVO.enc	Comprueba el hash SHA-256 y la firma digital
sudo respaldo_sgdm.sh descifrar ARCHIVO.enc	Verifica y, solo si está bien, descifra
sudo respaldo_sgdm.sh claves	Muestra dónde están las claves

6. Cómo se comprueba

Estas pruebas se pueden hacer en el servidor durante la defensa:

Qué se comprueba	Comando	Resultado esperado
Hash bcrypt de la aplicación	SELECT email, password FROM usuario LIMIT 3;	$2y$10$… (nunca la contraseña)
Hash SHA-512 del sistema	sudo grep '^sprieto:' /etc/shadow	sprieto:6…
Cifrado TLS	openssl s_client -connect IP_DEL_SERVIDOR:443 -brief </dev/null	Protocol version: TLSv1.3, Ciphersuite: TLS_AES_256_GCM_SHA384
Cifrado SSH	ssh -v -o BatchMode=yes USUARIO@IP_DEL_SERVIDOR exit 2>&1 | grep 'cipher:'	cipher: aes256-gcm@openssh.com

Respaldo cifrado	sudo sh -c 'head -c 8 "$(ls -t /var/backups/sgdm/bd/*.enc | head -n 1)"'; echo	Salted__ (no se ve el SQL)
Hash y firma de un respaldo	sudo respaldo_sgdm.sh verificar /var/backups/sgdm/bd/bd_sgdm_FECHA.sql.gz.enc	Hash SHA-256: OK. Firma digital: OK
Certificado firmado por la CA	sudo ssl_sgdm.sh estado	sgdm.crt: OK. En el navegador, el candado indica "emitido por Lidenskap CA SGDM"

Para mostrar que se detecta una modificación, se copia un respaldo con su .sha256 y su .sig y se le agrega un byte a la copia:
sudo mkdir -p /root/prueba
sudo sh -c 'cp /var/backups/sgdm/bd/bd_sgdm_FECHA.sql.gz.enc* /root/prueba/'
echo X | sudo tee -a /root/prueba/bd_sgdm_FECHA.sql.gz.enc >/dev/null
sudo respaldo_sgdm.sh verificar /root/prueba/bd_sgdm_FECHA.sql.gz.enc
El resultado es "Hash SHA-256: FALLA" y "Firma digital: FALLA", y descifrar se niega a abrir la copia. Si además se recalcula el .sha256 de la copia, el hash vuelve a dar OK pero la firma sigue fallando: solo la clave privada del servidor puede generar una firma válida.

BLOQUE D Plan de seguridad web basado en OWASP

Este plan define cómo nuestro proyecto cubre los diez riesgos del OWASP Top 10 (2021) y qué falta para llevarlo a producción. Partimos del código actual del repositorio, no de un sistema ideal: cada riesgo indica la medida ya implementada, el archivo donde está y la brecha a cerrar.

Objetivo. Proteger la confidencialidad de los datos personales de usuarios y atletas, la integridad de torneos, inscripciones y resultados, y la disponibilidad del sitio durante las competencias.

Alcance. Frontend (HTML, CSS y assets/js/main.js), backend PHP (app/api/*.php y app/src/), base MariaDB (db/) y sus dos despliegues: el entorno de desarrollo (SGDM App/Dockerfile y docker-compose.yml) y el servidor del ITS (Apache nativo en AlmaLinux 8 más una copia en Docker en el puerto 8080, /opt/sgdm/docker). El endurecimiento del sistema operativo (firewall, SSH, antivirus) se documenta en Administración de Sistemas Operativos (SEGURIDAD.md) y la criptografía en Ciberseguridad (CRIPTOGRAFIA.md); acá solo se citan cuando protegen a la aplicación web.

Activos a proteger.

- Cuentas y credenciales: tabla usuario, contraseñas con hash bcrypt y la cookie de sesión PHPSESSID.
- Datos personales: nombre, apellido y correo de usuarios y participantes.
- Datos de competencia: torneos, inscripciones, enfrentamientos, resultados y tabla de posiciones.
- Registro de auditoría (auditoria_registro), que prueba quién hizo cada cambio.
- Credenciales de la base en .env y claves de respaldo en /etc/sgdm/respaldos/.

1. Arquitectura del sistema

SGDM es una aplicación de tres capas: un frontend HTML y JavaScript, una API REST en PHP 8.3 sobre Apache y una base MariaDB 11.4. El frontend nunca habla con la base; toda escritura pasa por la API, que valida la sesión de servidor y el rol.

La sesión real vive en una cookie de PHP (Auth.php); lo que el frontend guarda en sessionStorage solo decide qué botones mostrar. Hay dos roles de cuenta, Administrador y Usuario público, y tres roles por torneo, Organizador, Árbitro y Participante, que la API consulta en vivo en vista_rol_torneo.

2. Metodología

El análisis sigue el OWASP Top 10 2021, la lista de referencia de riesgos en aplicaciones web, y usa el OWASP ASVS 4.0 nivel 1 como lista de verificación de los controles mínimos.

Prioridad	Criterio	Plazo de corrección
Alta	Explotable sin credenciales o compromete cuentas de administrador	Antes de publicar el sistema
Media	Requiere sesión o condiciones previas, o afecta solo a un usuario	Durante la siguiente iteración
Baja	Mejora de defensa en profundidad	Mantenimiento planificado

3. Análisis por riesgo OWASP Top 10

A01: Pérdida de control de acceso

Estado: parcial.

- Existente. Auth::requireLogin(), requireRole() y requireTorneoRole() (app/src/Auth.php) cortan con 401 o 403 antes de cualquier escritura. Hay dos roles de cuenta, Administrador y Usuario público; Organizador, Árbitro y Participante son roles por torneo que se leen en cada petición de vista_rol_torneo (RolesTorneo::tiene()) y se apagan solos cuando el torneo finaliza.
- Existente. Reglas de negocio en el servidor: solo el árbitro asignado carga el resultado de su partido (api/resultados.php); validar y registrar walkover es del Organizador; reabrir una ronda, borrar torneos y gestionar usuarios es solo del Administrador. Un Usuario público que crea un torneo deja una solicitud en estado solicitado, oculta al público hasta que el Administrador la apruebe.
- Existente. El autorregistro siempre crea un Usuario público y el panel solo asigna Administrador o Usuario público como rol de cuenta (lista blanca en api/usuario.php). Un Administrador no puede desactivarse ni quitarse el rol a sí mismo, las bajas son lógicas (estado = 'inactivo') y una invitación solo la puede aceptar la persona invitada.
- Existente (frontend). hasPermission() en main.js oculta opciones según el rol guardado en sessionStorage. Es solo estético: la decisión real la toma el servidor, que es lo correcto.

- SSRF (incluido en A01 desde 2025). No aplica hoy: el backend no hace peticiones salientes (solo lee php://input). Si se agregan logos desde una URL, se validarán con lista blanca de dominios y bloqueo de IP internas.
- Brecha. El rol de cuenta y el estado se toman de $_SESSION (Auth::current()): un usuario desactivado, o un Administrador pasado a Usuario público, conserva sus permisos hasta que su sesión vence.
- Brecha. No hay token anti-CSRF y ocho endpoints aceptan $_POST si el cuerpo no es JSON (login, register, usuarios, torneos, resultados, solicitudes, equipos, rondas). Hoy lo mitiga la cookie SameSite=Lax, que el navegador no envía en un POST desde otro sitio; quedan expuestos navegadores sin SameSite y otra aplicación servida en el mismo dominio o IP.
- Brecha. app/src/ está dentro de la raíz web: /src/Auth.php responde 200 (vacío, no muestra código; verificado).
- Brecha. La columna participante.perfil_publico no se usa: los nombres de todos los inscriptos se publican en api/torneo.php aunque el participante lo haya desactivado.
- Medidas. M-01 revalidar estado y rol contra la base; M-02 aceptar solo application/json y verificar Origin; M-03 bloquear src/ y el listado de directorios; M-04 respetar perfil_publico.

A02: Configuración de seguridad incorrecta

Estado: parcial.

- Existente (servidor). La imagen de /opt/sgdm/docker usa php.ini-production, expose_php = Off, ServerTokens Prod y ServerSignature Off; MariaDB no publica puertos y vive en una red interna; el contenedor web solo recibe las credenciales de sgdm_app; docker_sgdm.sh genera contraseñas aleatorias con openssl rand -hex 16. En el Apache nativo, firewalld solo abre 22, 80 y 443.
- Existente. Mínimo privilegio en la base: sgdm_app solo tiene SELECT, INSERT, UPDATE y DELETE, sin DDL, y sgdm_consulta es de solo lectura (db/02-users.sh).
- Existente. app/.htaccess envía Content-Security-Policy (script-src 'self', frame-ancestors 'none'), X-Content-Type-Options: nosniff, X-Frame-Options: DENY y Referrer-Policy.
- Brecha (alta). docker_sgdm.sh copia 03-seed-demo.sql a la base del servidor, así que la copia en Docker arranca con admin@lidenskap.com / admin123, contraseña publicada en el README del repositorio. La app no permite cambiar contraseñas, así que solo se corrige por SQL.

- Brecha. El entorno de desarrollo (SGDM App/Dockerfile) no tiene php.ini de producción: muestra errores con ruta y traza (ver A10) y anuncia X-Powered-By: PHP/8.3 (verificado). Su docker-compose.yml publica el puerto 3306 y pasa el .env completo, con las claves de root y de sgdm_admin, al contenedor web.
- Brecha. Apache tiene el listado de directorios activo por defecto y .htaccess no lo desactiva: /src/ y /api/ muestran la lista de archivos.
- Brecha. Faltan HSTS y Permissions-Policy.
- Medidas. M-03 Options -Indexes; M-05 quitar las cuentas demo del servidor; M-06 igualar la imagen de desarrollo a la del servidor; M-07 HSTS y Permissions-Policy.

A03: Fallas en la cadena de suministro de software

Estado: cubierto mayormente.

- Existente. La pila tiene versiones con soporte (PHP 8.3, MariaDB 11.4 LTS). No hay Composer, npm ni librerías de terceros: todo el PHP y el JavaScript es propio. En el servidor nativo dnf-automatic instala cada día las actualizaciones de seguridad con la firma GPG de AlmaLinux.
- Brecha. Las imágenes Docker usan etiquetas móviles (php:8.3-apache, mariadb:11.4): dos construcciones pueden traer versiones distintas, y nadie las escanea.
- Medidas. M-08 fijar versión de parche o digest, revisar cada mes y escanear con Trivy o Docker Scout.

A04: Fallas criptográficas

Estado: parcial.

- Existente. Contraseñas con password_hash(..., PASSWORD_DEFAULT) (bcrypt, costo 10 en PHP 8.3) y password_verify(); nunca se guarda texto plano ni se devuelve el hash al cliente. .env no se versiona.
- Existente. La cookie de sesión sale con HttpOnly, SameSite=Lax y Secure cuando la conexión es HTTPS (Auth::ensureStarted(); verificado).
- Existente. ssl_sgdm.sh crea la CA propia "Lidenskap CA SGDM", activa mod_ssl y redirige http a https (ver CRIPTOGRAFIA.md).
- Brecha (alta). Según el README de Ciberseguridad, ssl_sgdm.sh todavía no se ejecutó en el servidor real, y ni el script ni SSL.md están en el repositorio. Mientras tanto la contraseña del login y la cookie viajan sin cifrar.
- Brecha. La copia en Docker se publica solo por HTTP en el 8080, y Secure depende de detectar HTTPS en cada petición.
- Medidas. M-09 instalar HTTPS en el servidor y versionar el script; M-10 forzar Secure en producción y poner el 8080 detrás de HTTPS.

A05: Inyección (SQL, XSS y CSV)

Estado: cubierto.

- Existente (SQL). Toda consulta con datos del usuario usa sentencias preparadas de PDO, y Database.php desactiva la emulación (ATTR_EMULATE_PREPARES => false). Disciplina y formato pasan por listas blancas (Catalogos.php) y los ids se fuerzan a entero.
- Existente (XSS). main.js pasa por escapeHtml() todo dato del servidor que inserta con innerHTML (torneos, sede, descripción, inscriptos, equipos, auditoría, correo del menú); la ficha del torneo usa textContent. Además, la CSP script-src 'self' impide ejecutar un <script> o un onerror= aunque llegaran al HTML, y ninguna página tiene scripts en línea.
- Riesgo latente. Model::findBy(), create() y update() concatenan nombres de tabla y columna; hoy siempre vienen del código ('email', 'nombre_rol'…), nunca del usuario. El carrusel de torneos del inicio (renderTorneoCard()) interpola sin escapar, pero hoy no recibe datos del servidor.
- Brecha (baja). La exportación a CSV (downloadCsv()) no neutraliza celdas que empiezan con =, +, - o @: un nombre de torneo o un correo como =1+1@x.com (válido para FILTER_VALIDATE_EMAIL) se ejecuta como fórmula al abrir el archivo en Excel.
- Medidas. M-11 neutralizar fórmulas en CSV; M-12 regla de desarrollo (todo dato del servidor por escapeHtml() o textContent, columnas solo desde el código) y escapar el carrusel antes de conectarlo.

A06: Diseño inseguro

Estado: parcial.

- Existente. Las reglas de negocio viven en el servidor: las fases del torneo solo avanzan en orden (api/torneo.php), una ronda cerrada no se modifica, nadie puede tener dos papeles en el mismo torneo (RolesTorneo::yaVinculado()), las bajas son lógicas para no perder historial y las operaciones de varios pasos van en transacción.
- Brecha. api/torneos.php no valida largo máximo de nombre y sede (150 caracteres en la base), formato de fechaInicio ni tope de cupoMaximo; un valor fuera de rango termina en una excepción de la base (A10). Una disciplina personalizada crea una fila nueva en modulo_competencia aunque el torneo sea solo una solicitud.
- Brecha. El cupo se controla en dos pasos (cupoLleno() y después el INSERT): dos aprobaciones simultáneas pueden pasar el cupo.
- Brecha. La "Configuración del sistema" del panel, incluida "Auditoría obligatoria", se guarda en localStorage del navegador y no cambia nada en el servidor: aparenta un control que no existe.
- Medidas. M-13 validación completa en el servidor; M-14 cupo con bloqueo de fila; M-15 quitar o implementar en el servidor la configuración del panel.

A07: Fallas de autenticación

Estado: parcial.

- Existente. session_regenerate_id(true) al iniciar sesión y session.use_strict_mode evitan la fijación de sesión. El login responde "Correo o contraseña incorrectos" sin decir cuál falló, bloquea cuentas que no están activas y exige 8 caracteres en registro y alta. La cookie es HttpOnly, así que un script no puede robar la sesión.
- Existente (servidor). fail2ban tiene el jail sgdm-login: 10 logins fallidos (respuesta 401 de api/login.php) en 10 minutos bloquean la IP (SEGURIDAD.md).
- Brecha. Ese límite es solo por IP y solo en el Apache nativo: la copia en Docker y el entorno de desarrollo no tienen ninguno, y no hay límite por cuenta.
- Brecha. El registro responde "Ya existe una cuenta con ese correo", y el login tarda menos cuando el correo no existe (solo llama a password_verify() si lo encuentra): las dos cosas permiten saber qué correos tienen cuenta.
- Brecha. La sesión no vence de forma garantizada: PHP borra sesiones viejas al azar (gc_maxlifetime 1440 s) y no comprueba su edad al leerlas.
- Brecha. La política de contraseña solo exige largo, no existe cambio ni recuperación de contraseña y los administradores no tienen segundo factor.
- Medidas. M-16 límite por cuenta en la app; M-17 expiración por inactividad; M-18 respuestas sin enumeración; M-19 cambio de contraseña, rechazo de contraseñas comunes y 2FA para administradores.

A08: Fallas de integridad de software y datos

Estado: cubierto mayormente.

- Existente. El frontend no carga scripts de terceros y la CSP limita los orígenes a 'self' y Google Fonts.
- Existente. respaldo_sgdm.sh cifra cada respaldo con AES-256, guarda su hash SHA-256 y lo firma con RSA 4096; descifrar verifica hash y firma antes de restaurar, y una copia va a Google Drive.
- Brecha. La hoja de estilos de Google Fonts cambia según el navegador, así que no admite verificación con SRI.
- Brecha. Según el README de Ciberseguridad, el respaldo_sgdm.sh nuevo todavía no está instalado en el servidor real y quedan respaldos viejos sin cifrar, también en Drive. No hay prueba periódica de restauración.
- Medidas. M-20 instalar el respaldo nuevo y probar una restauración por mes; M-21 servir las fuentes desde el propio sitio.

A09: Fallas de registro y monitoreo

Estado: parcial.

- Existente. Auditoria::registrar() (app/src/Auditoria.php) guarda usuario, acción, tabla, detalle, fecha e IP de cada acción que cambia datos: inicios de sesión (también los fallidos), altas, ediciones y bajas de usuarios, torneos, resultados, rondas y el motivo de cada reapertura. El panel muestra el registro con búsqueda y exportación (api/auditoria.php, solo Administrador).
- Brecha. Las denegaciones 401 y 403 (Auth::deny()) no quedan registradas, y no hay alertas: nadie se entera de una ráfaga de logins fallidos o del alta de un Administrador hasta que mira el panel.
- Brecha. sgdm_app tiene UPDATE y DELETE sobre auditoria_registro: la misma cuenta que escribe el registro lo puede borrar.
- Brecha. Los logins fallidos con correos inexistentes se registran sin límite, así que se puede llenar la tabla; el panel solo muestra los 1000 eventos más recientes y no muestra el campo detalle.
- Medidas. M-22 registrar 401 y 403 y alertar; M-23 auditoría de solo inserción para sgdm_app.

A10: Falsificación de peticiones del lado del servidor

Estado: parcial.

- Existente. Los endpoints devuelven JSON con códigos coherentes (400, 401, 403, 404, 405, 409) y las operaciones de varios pasos usan transacciones, que PDO revierte si el script termina sin commit().
- Brecha. No hay manejador global de excepciones. Si la base no responde o rechaza un dato (fecha inválida, nombre de 151 caracteres), la PDOException no se captura. En el entorno de desarrollo la respuesta es HTTP 200 con un "Fatal error" en HTML, la ruta /var/www/html/src/Database.php y la traza completa (verificado); en el servidor, php.ini-production lo convierte en un 500 vacío. En los dos casos el frontend espera JSON y no muestra ningún mensaje.
- Medidas. M-24 manejador global que registre el error y devuelva un 500 JSON genérico; M-06 y M-13 reducen los casos.

4. Matriz resumen de riesgos

Riesgo OWASP 2025	Estado	Probabilidad	Impacto	Prioridad	Medidas
A02 Configuración incorrecta	parcial	Alta	Alto	Alta	M-03, M-05, M-06, M-07
A04 Fallas criptográficas	parcial	Media	Alto	Alta	M-09, M-10
A01 Control de acceso	parcial	Media	Alto	Alta	M-01, M-02, M-03, M-04
A07 Autenticación	Parcial	Media	Alto	Media	M-16, M-17, M-18, M-19
A10 Condiciones excepcionales	Parcial	Media	Medio	Media	M-24
A06 Diseño inseguro	Parcial	Media	Medio	Media	M-13, M-14, M-15
A09 Registro y alertas	Parcial	Media	Medio	Media	M-22, M-23
A08 Integridad	cubierto mayormente	Baja	Medio	Baja	M-20, M-21
A03 Cadena de suministro	cubierto mayormente	Baja	Medio	Baja	M-08
A05 Inyección	Cubierto	Baja	Medio	Baja	M-11, M-12

6. Plan de acción

Id	Medida	Dónde se aplica	Responsable	Fase
M-05	No copiar 03-seed-demo.sql al servidor; si ya se cargó, cambiar o desactivar las cuentas demo	docker_sgdm.sh, base del servidor	Infraestructura	1
M-09	Ejecutar ssl_sgdm.sh instalar en el servidor real, importar la CA en las PC de la defensa y subir ssl_sgdm.sh y SSL.md al repositorio	Servidor, repositorio	Infraestructura	1
M-01	Revalidar estado y rol de la cuenta contra la base en cada petición protegida	src/Auth.php	Backend	1
M-03	Options -Indexes en .htaccess y Require all denied en src/.htaccess	app/.htaccess, app/src/.htaccess	Backend	1
M-06	Igualar la imagen de desarrollo a la del servidor: php.ini-production, expose_php = Off, ServerTokens Prod, sin publicar 3306 y solo las variables de sgdm_app en el contenedor web	SGDM App/Dockerfile, docker-compose.yml	Infraestructura	1
M-24	Manejador global de excepciones: registrar en error_log y devolver 500 con JSON genérico	src/autoload.php	Backend	1
M-02	Aceptar solo Content-Type: application/json (quitar el respaldo a $_POST) y rechazar con 403 un Origin ajeno	api/*.php, src/Auth.php	Backend	1
M-16	Límite por cuenta en la app: 5 fallos en 15 minutos devuelven	api/login.php, api/register.php	Backend	1

	429, además de fail2ban			
M-20	Instalar el respaldo_sgdm.sh nuevo, guardar cifrado.clave fuera del servidor, borrar respaldos sin cifrar y probar una restauración por mes	Servidor, Google Drive	Infraestructura	1
M-10	Forzar session.cookie_secure = 1 en producción y poner el 8080 detrás de un proxy con HTTPS (o no exponerlo)	src/Auth.php, servidor	Infraestructura	2
M-13	Validación en el servidor: largos según la base, fecha con DateTime::createFromFormat('Y-m-d'), cupo entre 2 y un tope, disciplina personalizada creada solo al aprobar	api/torneos.php y demás endpoints de escritura	Backend	2
M-17	Cerrar la sesión tras 30 minutos sin actividad, comprobado en Auth::current()	src/Auth.php	Backend	2
M-18	Mensaje neutro en el registro y password_verify() contra un hash ficticio cuando el correo no existe	api/register.php, api/login.php	Backend	2
M-22	Registrar las respuestas 401 y 403 y alertar ante ráfagas de logins fallidos o altas de Administrador	Auth::deny(), Auditoria.php, monitoreo_sgdm.sh	Backend	2
M-23	Dejar a sgdm_app solo SELECT e INSERT sobre auditoria_registro (permisos por tabla)	db/02-users.sh	Infraestructura	2
M-11	Prefijar ' a las celdas CSV que empiezan con =, +, - o @	main.js (downloadCsv)	Frontend	2

M-15	Quitar del panel la "Configuración del sistema" o guardarla y aplicarla en el servidor	panel.html, main.js	Frontend	2
M-07	Agregar HSTS y Permissions-Policy una vez activo HTTPS	app/.htaccess o virtual host	Infraestructura	2
M-19	Cambio de contraseña, rechazo de contraseñas comunes y 2FA (TOTP) para administradores	Nuevo api/password.php, api/login.php	Backend	3
M-14	Controlar el cupo dentro de la transacción con SELECT … FOR UPDATE	RolesTorneo::cupoLleno(), api/solicitudes.php	Backend	3
M-04	Respetar perfil_publico en las listas públicas de inscriptos	api/torneo.php, api/rondas.php	Backend	3
M-12	Escapar renderTorneoCard() antes de conectarlo a la API y mantener la regla escapeHtml()/textContent	main.js, revisión de código	Frontend	3
M-08	Fijar imágenes a versión de parche o digest, revisarlas cada mes y escanearlas con Trivy o Docker Scout	Dockerfile, docker-compose.yml	Infraestructura	3
M-21	Servir las fuentes desde assets/fonts/ y quitar Google Fonts de la CSP	HTML, styles.css, .htaccess	Frontend	3

5. Pruebas de seguridad y monitoreo continuo

Cada medida se da por cerrada solo cuando pasa su caso de prueba; estos casos se suman al plan de testing de la entrega (PHPUnit para el backend, Cypress para el flujo en el navegador).

Caso	Riesgo	Pasos	Resultado esperado	Hoy	Herramienta
CS-01	A05	Enviar ' OR '1'='1 como correo en el login	401 "Correo o contraseña incorrectos."	Pasa	Cypress, sqlmap
CS-02	A05	Crear un torneo llamado <img src=x onerror=alert(1)>, aprobarlo y abrir torneos.html, el detalle y el panel	El texto se ve literal y no aparece la alerta	Pasa	Cypress
CS-03	A01	POST api/torneos.php sin sesión y con sesión de Usuario público	401 sin sesión; con Usuario público, 200 con solicitud: true y el torneo no aparece en GET api/torneos.php	Pasa	PHPUnit, curl
CS-04	A01	DELETE api/usuario.php?id= con el propio id del Administrador	400 "No podés desactivar tu propia cuenta."	Pasa	PHPUnit
CS-05	A01	Con un Usuario público, POST api/resultados.php en un partido que no arbitra	403 "Solo el árbitro asignado…"	Pasa	PHPUnit

CS-06	A01	Desde otro origen, enviar un formulario HTML a api/usuarios.php con un Administrador logueado	No se crea la cuenta: 401 hoy (el navegador no manda la cookie Lax); 403 tras M-02 aunque llegue	Pasa	Navegador , Burp Suite
CS-07	A01	Desactivar un usuario con sesión abierta y repetir su última acción	401	Falla (M-01)	PHPUnit
CS-08	A07	6 logins fallidos seguidos con el mismo correo	El sexto devuelve 429	Falla (M-16); en el servidor nativo fail2ban bloquea la IP al décimo	Cypress, fail2ban-client status sgdm-login
CS-09	A07	Registrarse con una contraseña de 7 caracteres	400 "La contraseña debe tener al menos 8 caracteres."	Pasa	PHPUnit
CS-10	A04, A02	Pedir http:// y revisar las cabeceras de https://…/api/login.php	Redirección a https://; Set-Cookie con Secure; HttpOnly; SameSite=Lax; CSP, HSTS, X-Frame-Options y nosniff	Falla (M-09, M-07)	curl, OWASP ZAP
CS-11	A02, A01	GET /src/, /api/ y /src/Auth.php	403 o 404, sin listado de archivos	Falla (M-03)	curl
CS-12	A10	Con sesión, POST api/torneos.php con fechaInicio: "abc" o un nombre de 151 caracteres	400 con mensaje claro (M-13) o 500 JSON genérico, sin rutas ni trazas	Falla (M-24)	curl

CS-13	A02	Iniciar sesión en el servidor con admin@lidenskap.com / admin123	401	Falla en la copia Docker (M-05)	curl
CS-14	A09	Un login fallido y abrir Panel → Auditoría	Aparece "Inicio de sesión fallido" con fecha e IP	Pasa	Cypress

BLOQUE C Políticas de seguridad de usuario

Estas políticas definen las reglas necesarias que debe cumplir cada persona que tenga una cuenta en el SGDM o en su servidor. Su objetivo es proteger las cuentas, los datos personales de los participantes y la integridad de los resultados oficiales de cada torneo.

La política alcanza dos capas de usuarios independientes:

- Usuarios de la aplicación web: Administrador general, Organizador de torneo, Árbitro, Participante, Usuario público y visitantes sin cuenta.
- Cuentas técnicas del servidor: cuentas del sistema operativo AlmaLinux (administradores del equipo y usuario de servicio) y usuarios de la base de datos MariaDB.

Ningún usuario del sistema operativo representa un rol de la aplicación, ni viceversa. Las reglas de esta política complementan las medidas técnicas ya documentadas (validaciones, hashing de contraseñas, firewall, fail2ban y SSL) y se apoyan en la Ley N.º 18.331 de Protección de Datos Personales.

2. Roles y tipos de usuario

La app tiene dos roles globales (Administrador general y Usuario público) y tres roles por torneo (Organizador, Árbitro y Participante). Un rol por torneo solo vala dentro de ese torneo y se revoca automáticamente cuando el torneo pasa a finalizado o cancelado; la cuenta vuelve a ser Usuario público sin perder su historial.

2.1 Roles de la aplicación web

Rol	Alcance	Cómo se obtiene	Puede	No puede
Administrador general (ADMIN)	Global	Lo crea otro Administrador desde el panel	Gestionar usuarios, aprobar solicitudes de torneo, gestionar todos los torneos y resultados, ver la auditoría y la configuración	Quitarse su propio rol ni desactivar su propia cuenta
Organizador de torneo (ORGANIZADOR)	Por torneo	Solicitud de creación de torneo aprobada por el Administrador, o asignación directa	Gestionar su torneo, aprobar árbitros e inscripciones de ese torneo, cargar resultados, ver reportes	Operar torneos que no organiza, gestionar usuarios, ver la auditoría
Árbitro (ARBITRO)	Por torneo	Solicitud aprobada por el Organizador del torneo o el Administrador	Cargar y validar resultados del torneo asignado	Cambiar la estructura del torneo, las inscripciones u otros torneos
Participante (PARTICIPANTE)	Por torneo	Inscripción confirmada por el Organizador o el Administrador	Ver y editar su perfil, consultar su calendario y resultados	Modificar resultados, torneos o datos de otros
Usuario público (PUBLICO)	Global (por defecto)	Autorregistro con correo y contraseña	Consultar la información pública, solicitar crear un torneo, arbitrar o participar	Cualquier acción de gestión
Visitante	Sin cuenta	No requiere registro	Consultar calendarios, resultados y posiciones públicas	Todo lo que requiera iniciar sesión

2.2 Cuentas técnicas del servidor

Cuenta	Tipo	Privilegios	Restricciones
sprieto, frodriguez, gvisos, jvillena	Administradores del equipo (SO)	sudo a través del grupo sgdm_admins; miembros de sgdm_devs	Personales e intransferibles; único grupo autorizado a entrar por SSH
svc_sgdm	Usuario de servicio (SO)	Dueño de los archivos de la aplicación (permisos 750)	Sin sudo y sin shell interactiva (/sbin/nologin)
root	Superusuario (SO)	Todos	Prohibido el ingreso directo por SSH (PermitRootLogin no)
sgdm_admin	Base de datos	Todos los privilegios sobre la base sgdm	Solo para mantenimiento y respaldos
sgdm_app	Base de datos	SELECT, INSERT, UPDATE, DELETE	Sin permisos de estructura (DDL)
sgdm_consulta	Base de datos	SELECT sobre las tablas publicables	Sin escritura ni acceso a usuario o auditoria_registro

3. Contraseñas y autenticación

Toda contraseña de la aplicación debe tener al menos 10 caracteres con: mayúsculas, minúsculas, números y símbolos, y se guarda solo como hash bcrypt. Las cuentas del servidor usan contraseñas temporales aleatorias que deben cambiarse en el primer ingreso.

3.1 Cuentas de la aplicación

Regla	Valor
Longitud	Mínimo 10 caracteres, máximo 72 (límite de bcrypt)
Composición	Al menos una mayúscula, una minúscula, un número y un símbolo
Contraseñas prohibidas	Las que contienen el nombre, el apellido o el correo del usuario, y las de uso común (por ejemplo 12345678 o Lidenskap2026)
Almacenamiento	Hash bcrypt con sal aleatoria (password_hash con PASSWORD_DEFAULT); nunca texto plano, MD5 ni SHA-1
Mensaje ante error	Siempre el mismo: “Correo o contraseña incorrectos.”, para no revelar qué correos están registrados
Bloqueo por cuenta	5 intentos fallidos seguidos bloquean la cuenta durante 15 minutos
Bloqueo por dirección IP	fail2ban (jail sgdm-login): 10 fallos en 10 minutos bloquean la IP durante 1 hora
Contraseña asignada por el Administrador	Se comunica por un canal distinto al correo de la cuenta y debe cambiarse en el primer inicio de sesión
Historial	No se pueden reutilizar las últimas 3 contraseñas
Vencimiento	Cada 90 días para Administrador general y Organizador; el resto, solo ante sospecha de compromiso

Restablecimiento	Solo lo realiza el Administrador, tras verificar la identidad del titular; genera una contraseña temporal de cambio obligatorio

3.2 Cuentas del servidor

Regla	Valor
Contraseña inicial	Aleatoria, generada con openssl rand -base64 12 (16 caracteres) por gestion_usuarios.sh
Primer ingreso	Cambio obligatorio (chage -d 0)
Almacenamiento	/etc/shadow con hash SHA-512 (predeterminado en AlmaLinux)
Acceso SSH	Solo el grupo sgdm_admins; root sin acceso; 3 intentos por conexión (MaxAuthTries 3) y 30 segundos para autenticarse
Fuerza bruta en SSH	fail2ban (jail sshd): 5 fallos en 10 minutos bloquean la IP durante 1 hora
Usuarios de MariaDB	Una contraseña distinta por usuario, guardada en el archivo .env del servidor; los valores de ejemplo changeme se reemplazan antes de producción y nunca se suben al repositorio
Próxima mejora	Ingreso por SSH solo con llaves (PasswordAuthentication no)

3.3 Obligaciones de todo usuario

1. No compartir la contraseña con nadie, ni siquiera con otro integrante del equipo o con un docente.
2. No reutilizar una contraseña usada de otro lado(correo, redes sociales, juegos).

3. Cambiar la contraseña de inmediato y avisar al Administrador ante cualquier sospecha de que otra persona la conoce.
4. Lidenskap nunca pide contraseñas por correo, mensaje ni teléfono: cualquier pedido de ese tipo es un intento de phishing y debe reportarse.

4. Ciclo de vida de las cuentas

Un Usuario público o Administrador, recibe roles de torneo solo por solicitud aprobada y nunca se borra físicamente: la baja es lógica para conservar el historial de competencias.

[embed: node/24ecd78c-b70c]

Una solicitud rechazada no cambia la cuenta. El Administrador puede pasar a Inactiva una cuenta Activa o Suspendida, pero nunca la suya.

4.1 Alta

- Autorregistro: nombre, apellido, correo válido y una contraseña. La cuenta queda como Usuario público y el titular acepta los Términos y la Política de privacidad.
- Creación por el Administrador: solo él crea cuentas de Administrador o asigna un rol en un torneo vigente (planificado o en curso) al crear la cuenta.
- Una persona, una cuenta: un correo identifica a una sola cuenta; está prohibido crear cuentas falsas, duplicadas o automatizadas.
- Menores de edad: se aplican las condiciones del punto 7.

4.2 Roles por torneo

- Crear un torneo lo aprueba solo el Administrador; arbitrar o participar lo aprueba el Organizador de ese torneo o el Administrador.
- El rol se revoca solo cuando el torneo pasa a finalizado o cancelado, sin que el usuario tenga que volver a iniciar sesión.

4.3 Modificación, suspensión y baja

- Solo el Administrador modifica el nombre, el correo o el rol global de otra cuenta; antes de cambiar un correo verifica la identidad del titular.
- Ningún Administrador puede quitarse su propio rol, para que el sistema nunca quede sin administración.
- Suspensión: ante sospecha de cuenta comprometida o incumplimiento de esta política, hasta que se resuelva el caso.
- inactividad: a pedido del titular, al dejar un integrante el proyecto o tras 12 meses sin uso.
- Toda alta, edición, suspensión o baja queda en el registro de auditoría con fecha, hora, autor e IP.

4.4 Cuentas del servidor

- Solo un miembro de sgdm_admins crea cuentas, siempre con gestion_usuarios.sh, y cada cuenta es personal.
- Cuando un integrante deja el equipo, su cuenta se bloquea el mismo día (usermod -L y shell /sbin/nologin) y se lo retira de sgdm_admins.
- Cada tres meses se revisa quién integra sgdm_admins y qué cuentas tienen shell interactiva.

5. Control de acceso y sesiones

Cada permiso se verifica en el servidor en cada petición; ocultar un botón en la interfaz nunca cuenta como control de acceso.

5.1 Principios

- Mínimo privilegio: cada usuario recibe solo el rol que necesita y solo en el torneo donde lo necesita.
- Verificación en el backend: todo endpoint que modifica datos exige sesión y rol (Auth::requireLogin, requireRole y requireTorneoRole). Sin sesión responde 401 y sin permiso, 403.
- Roles consultados en vivo: los roles por torneo se leen de la base en cada petición. El rol guardado en el navegador (sessionStorage) solo sirve para dibujar la interfaz y nunca autoriza una acción.
- Separación de funciones: quien pide organizar un torneo no puede aprobar su propia solicitud; solo lo hace el Administrador.
- Auditoría protegida: solo el Administrador consulta el registro de auditoría, y nadie lo modifica ni lo borra desde la aplicación.

5.2 Sesiones

Regla	Valor
Identificador	Cookie de sesión de PHP, regenerada en cada inicio de sesión (session_regenerate_id) para evitar la fijación de sesión
Atributos de la cookie	HttpOnly, Secure y SameSite=Strict
Transporte	Solo por HTTPS; toda petición HTTP se redirige a HTTPS
Cierre por inactividad	30 minutos para Administrador, Organizador y Árbitro; 2 horas para el resto
Duración máxima	8 horas; después se vuelve a pedir la contraseña
Cierre de sesión	Destruye la sesión en el servidor y borra los datos guardados en el navegador
Protección CSRF	Token anti-CSRF en toda petición que modifica datos (POST, PUT, PATCH, DELETE)
Suspensión o baja	Invalida de inmediato todas las sesiones abiertas de esa cuenta

6. Uso aceptable y responsabilidades

Cada usuario responde por todo lo que se haga con su cuenta y solo puede usar la app para organizar, arbitrar, disputar o seguir competencias.

6.1 Responsabilidades por rol

Rol	Responsabilidades
Administrador general	Usar la cuenta de Administrador solo para tareas de administración; revisar la auditoría al menos una vez por semana; aprobar solicitudes solo tras verificar al solicitante; aplicar esta política sin excepciones
Organizador de torneo	Aprobar solo inscripciones y árbitros legítimos; mantener actualizados fechas y estados del torneo; usar los datos de los participantes solo para ese torneo
Árbitro	Cargar los resultados tal como ocurrieron y apenas termine cada enfrentamiento; informar al Organizador cualquier error o reclamo
Participante y Usuario público	Dar datos verídicos y mantenerlos al día; proteger su contraseña; respetar las reglas de juego limpio de cada disciplina
Lidenskap (servidor)	Usar sudo solo cuando haga falta; no copiar la base de datos ni el archivo .env fuera del servidor salvo en los respaldos autorizados; documentar cada cambio de configuración

6.2 Conductas prohibidas

- Usar la cuenta de otra persona o prestar la propia.
- Crear cuentas falsas o automatizar registros, inscripciones o inicios de sesión con scripts o bots.
- Alterar resultados, posiciones o inscripciones de forma fraudulenta.
- Ingresar código, scripts o contenido ofensivo en nombres de torneos, equipos o perfiles.
- Copiar, publicar o vender datos personales de otros usuarios.
- Escanear, probar vulnerabilidades o saturar el servidor sin autorización escrita de la empresa .

7. Protección de datos personales

7.1 Datos que se guardan

Dato	Para qué se usa	Quién lo ve
Nombre y apellido	Identificar la cuenta y al participante	Administrador, Organizador del torneo; el público solo si el perfil es público
Correo electrónico	Iniciar sesión y contactar al titular	Administrador y el propio titular
Contraseña	Autenticar al usuario	Nadie: solo se guarda su hash bcrypt
Fecha de nacimiento	Ubicar al participante en su categoría	Administrador y Organizador del torneo; nunca se publica
Alias y resultados	Mostrar posiciones y calendarios	Público
Registro de auditoría (acción, fecha, hora, IP)	Seguridad y trazabilidad de cambios	Solo el Administrador; se conserva 12 meses

7.2 Menores de edad

- Un menor de 18 años puede inscribirse solo con la autorización de su madre, padre o tutor, que el Organizador debe tener antes de confirmar la inscripción.
- El perfil de un menor queda privado por defecto: el público ve su alias y sus resultados, nunca su nombre completo, correo ni fecha de nacimiento.
- Las fotos o datos de contacto de menores no se publican en ninguna sección del sitio.

7.3 Derechos del titular

Todo titular puede pedir acceso, rectificación, actualización o supresión de sus datos escribiendo a empresalidenskap@gmail.com o por la sección Contacto. El equipo responde en un máximo de 5 días hábiles.

Como los resultados oficiales deben conservarse, la supresión se cumple dando de baja la cuenta y anonimizando sus datos: el nombre se reemplaza por “Participante eliminado” y se borran el correo y la fecha de nacimiento.

8. Reporte de incidentes de seguridad

Todo usuario que detecte un problema de seguridad debe reportarlo dentro de las 24 horas. Si hay datos personales comprometidos, la empresa avisa a la URCDP en un máximo de 72 horas, como exige el Decreto N.º 64/020.

Qué se reporta: una contraseña expuesta, actividad que el titular no reconoce en su cuenta, resultados o inscripciones alterados, mensajes que piden credenciales, pantallas que muestran datos de otra persona y alertas de fail2ban o ClamAV en el servidor.

Cómo se reporta: un correo a empresalidenskap@gmail.com con el asunto “Incidente de seguridad”, indicando qué pasó, cuándo y qué cuenta está involucrada. Quien reporta no debe investigar por su cuenta ni borrar evidencia.

Qué hace el equipo:

1. Registrar el reporte con fecha, hora y persona que lo realizó.
2. Contener: suspender las cuentas afectadas, cerrar sus sesiones, bloquear las IP de origen con fail2ban o firewalld y cambiar las contraseñas expuestas.
3. Investigar con el registro de auditoría, los logs de Apache (sgdm-access.log, sgdm-ssl-access.log), /var/log/sgdm_seguridad.log y el journal del sistema.
4. Recuperar: corregir la falla y, si hubo datos alterados, restaurar el último respaldo verificado.
5. Comunicar el incidente a los usuarios afectados y, si incluye datos personales, a la URCDP dentro de las 72 horas.
6. Cerrar con un informe de causa, impacto y medidas tomadas, y actualizar esta política si hace falta.

9. Concientización, cumplimiento y revisión

Todo usuario acepta esta política al registrarse, su incumplimiento se sanciona según la gravedad y el documento se revisa una vez por año o ante cualquier cambio importante del sistema.

9.1 Concientización

- El formulario de registro enlaza los Términos, la Política de privacidad y un resumen de esta política.
- Los integrantes de Lidenskap y cada Organizador leen la política completa antes de recibir su rol.
- Todo acceso por SSH muestra un aviso legal de acceso restringido (/etc/issue.net).

9.2 Sanciones

Gravedad	Ejemplos	Medida
Leve	No cerrar sesión en un equipo compartido; datos personales desactualizados	Advertencia por correo
Grave	Prestar la cuenta; intentar acceder a torneos ajenos; crear cuentas duplicadas	Suspensión de la cuenta y pérdida de los roles de torneo
Muy grave	Alterar resultados; copiar o publicar datos personales; atacar el servidor	Baja definitiva, descalificación y, si corresponde, denuncia ante la autoridad